



AI System Design Project - A13104

Project Masar

Dr. Hawazin Badawi

Project Members' Task Distribution Matrix

Name	ID	Group	Task
Maria Alzahery	445006784	2	2.3
Ghadeer Sami	44511798	2	Chapter 3
Manar Ahmed	445006780	3	Chapter 1 + 2.1
Ghala Shahir	445006326	2	2.4 + Chapter 4
Retaj Alhazmi	445006594	1	2.2 + 2.5

Table of Contents

Chapter 1: Introduction	5
1.1 Abstract	5
Chapter 2: AI Lifecycle	6
2.1 Scoping: Introduction & Problem Definition	6
2.1.1 Problem Statement & Background Context	6
2.1.2 Target User Group & System Scope	6
2.1.3 Success Metrics & Key Performance Indicators (KPIs)	7
2.1.4 Feasibility & Initial Risk Assessment	8
Table 1: Risk Assessment and Mitigation Strategies	9
2.1.5 Ethical, Legal, and Societal Considerations	10
2.2 Data: Data Collection, Preparation, and Governance	11
2.2.1 Data Sourcing & Justification	11
2.2.2 Data Collection & Ingestion Pipeline	11
2.2.3 Data Cleaning & Preprocessing Protocol	12
2.2.4 Data Governance & Privacy	13
2.3 Modeling: Model Development, Baseline Selection, and Iterative Training	14
2.3.1 Baseline Model Choice & Justification	14
2.3.2 Algorithm Selection & Model Architecture	15
Table 2: TransGeo Architecture Configuration Summary	17
2.3.3 Modular Code & Version Control Execution	17
2.3.4 Model Training & Hyperparameter Tuning	18
Table 3: Complete Hyperparameter Configuration for TransGeo Training	19
2.3.5 Experimentation Plan & Iterative Improvement	20
Table 4: Average Training Loss Across Training Epochs	20
Table 5: FAISS Retrieval Performance and Inference Latency Results	21
Table 6: Iterative Improvements Applied During MASAR Development	22
2.4 Deployment: System Design, Serving, and Infrastructure	22
2.4.1 Comprehensive Logical Architecture	22
Figure 1: MASAR Secure Login Interface Prototype	24
Figure 2: MASAR Operational Dashboard Prototype	25
Figure 3: MASAR Prediction and XAI Explanation Interface Prototype	25
2.4.2 Production Consideration	26
2.4.3 Deployment & Serving Strategy	26
2.4.4 Continuous Integration & Continuous Deployment (CI/CD)	27
2.5 Monitoring: System Maintenance, Telemetry, and Testing Hooks	28
2.5.1 Monitoring/Logging Hooks	28
2.5.2 Error Handling & Test Coverage	28
2.5.3 Model Drift & Update Strategy	29
2.5.4 Disaster Recovery & System Reliability	30
Chapter 3: Results, Evaluation & Performance Analysis	30
3.1 Retrieval Scalability Analysis	30

Table 7: MASAR Recall@K Evaluation Across Search Scales	31
3.2 Load and Computational Efficiency Test Results	31
Table 8: Computational Inference Latency and Throughput Under Load	31
3.3 Robustness and Environmental Stress Testing	32
Table 9: Stress Test Scenarios and Target Operational Simulations	32
Table 10: MASAR Recall@1 Performance Under Stress Conditions	33
Figure 4: Visual Specification & Caption	34
3.4 Explainable AI (XAI) and Visual Verification	35
Figure 5: Saliency Map	35
Chapter 4: Conclusion, Reflection & Future Work	36
4.1 System Limitations	36
4.2 Reflection on Lessons Learned	36
4.3 Recommendations for Future Improvements	37
5. References	38

Chapter 1: Introduction

1.1 Abstract

Autonomous systems, such as drones, rely completely on the Global Positioning System (GPS) for navigation. However, the exposure of these signals to degradation, jamming, or complete loss strips the aircraft of their ability to determine their locations accurately, thereby threatening the continuity of their missions, in addition to the current alternative solutions lack of transparency and reliability. This project (MASAR) aims to develop an innovative navigation system that relies on computer vision as a safe alternative in the absence of GPS signals. The proposed system captures live images of the location and processes them via the cloud using deep learning algorithms, matching them with reference satellite images to determine coordinates with high accuracy. Standing out from conventional solutions, the system integrates Explainable Artificial Intelligence (XAI) techniques to clarify the reasons and nature of the location prediction, which enhances the system's reliability and transparency. This project is expected to contribute to raising the efficiency of drones and enabling them to execute exploration operations in critical and hazardous environments, supporting emergency response teams, search and rescue missions, and security sectors that require autonomous navigation and high reliability.

Chapter 2: AI Lifecycle

2.1 Scoping: Introduction & Problem Definition

2.1.1 Problem Statement & Background Context:

Traditional navigation systems mainly rely on Global Positioning System (GPS) signals to estimate the position of vehicles and drones. However, GPS-based navigation can become unreliable or unavailable in many real-world situations, such as urban canyons, tunnels, indoor areas, construction and excavation sites, and regions affected by signal jamming or spoofing [1]. This poses a major challenge for autonomous systems that require continuous, accurate, and real-time localization, such as drones, to operate safely and effectively.

Current solutions lack transparency in AI models, causing them to function as a "black box", which is critical in drone navigation, where operators need to understand the visual basis the system relied on for localization to build trust, mitigate risks, and achieve operational objectives.

The proposed project, MASAR, aims to design an intelligent navigation system based on computer vision instead of fully depending on GPS signals. The system will use visual inputs from onboard cameras to recognize the surrounding environment, estimate movement, and determine the position with the highest possible accuracy even when the GPS signal is weak, blocked, or intentionally degraded [1][2].

The primary objective of the project is to develop a vision-based localization and navigation solution by integrating Transformer technologies through the TransGeo model, which can provide reliable positioning supported by Explainable AI (XAI) in GPS-denied or GPS-degraded environments. To achieve this, the system must be capable of capturing visual data represented by drone images as inputs, extracting long-range critical features from the environment via self-attention mechanisms, tracking these features over time, and estimating the current location with acceptable accuracy.

The key requirements of the system include high-accuracy visual localization and efficient cloud-based processing in the shortest possible time, while monitoring and balancing latency challenges to ensure real-time and rapid responsiveness, with robustness against lighting variations, scene changes, and challenging environmental conditions, and compatibility with autonomous platforms such as drones, in addition to providing transparent visual attention maps Saliency Visualization and importance maps to highlight the landmarks and building rooftops that the model relied upon to make its navigational decision.

2.1.2 Target User Group & System Scope:

The target user group for the MASAR system includes organizations and operators who need a reliable navigation system in environments where GPS signals may be lost, blocked, or jammed. This primarily includes drone operators, autonomous vehicle developers, emergency

response teams, search and rescue for missing persons, security and defense sectors such as the Ministry of Defense, and industrial inspection teams working in complex or hazardous environments.

For example, drone operators can use the system for inspection, surveillance, and monitoring tasks in areas where GPS performance is poor. Emergency teams can benefit from it during search and rescue missions in disaster zones or damaged infrastructure. In addition, researchers and engineers working on autonomous systems can use the project as a foundation for developing more reliable and advanced navigation technologies.

The system scope includes capturing real-time images of the location and processing them via the cloud, where the TransGeo model was selected to be the model that analyzes the images, extracts features, and predicts the location, in addition to displaying the saliency map and importance features to show supervisors the reasoning behind the prediction. The model is trained on the University-1652 dataset and tested on a simulation environment, where no real drone hardware is utilized.

2.1.3 Success Metrics & Key Performance Indicators (KPIs):

To systematically evaluate the performance of the MASAR system, the evaluation framework is divided into two main categories:

Operational and Business KPIs:

- **System Reliability:** Evaluates software stability through continuous operation within the simulation to ensure the absolute absence of crashes or total tracking loss during long-duration flights [3].
- **Inference Time & Latency:** Measures the delay between capturing an image and producing a location estimate, ensuring that cloud-based processing maintains real-time responsiveness [3].
- **Robustness:** Ensures high system efficiency and stable navigation under varying environmental conditions such as lighting variations, shadows, or fog within the simulation [3].

Machine Learning Performance Criteria:

- **Localization Accuracy (ATE):** Measures the absolute trajectory error to calculate the precise distance between the model's predicted location and the ground-truth location, aiming for an error within 1–2 meters [4].
- **Drift Error (RPE):** Tracks relative pose error to monitor and mitigate the accumulation of feature matching errors over time during continuous motion [4].
- **Feature Tracking Stability:** Measures the algorithmic capability to maintain tracking continuity across consecutive simulation frames, preventing sudden spatial jumps [3].

2.1.4 Feasibility & Initial Risk Assessment:

The technical feasibility of the MASAR system is based on the integration of proven computer vision algorithms and available software frameworks:

- **Software Ecosystem:** The project will be developed using Python and the OpenCV library [5]. This ecosystem provides ready-to-use modules for Visual Odometry [6] and Template Matching, allowing us to focus on the logic of GPS-denied navigation rather than building basic image processing tools from scratch.
- **Feature Extraction & Matching:** We will utilize the TransGeo model, which leverages a Vision Transformer (ViT) architecture [7]. This model is characterized by its high capability to capture long-range spatial dependencies and extract rich, distinctive features from images via self-attention mechanisms. This makes it ideal for matching between the drone perspective and the satellite perspective, while remaining robust against scale and rotation changes to recognize the distinctive and clear terrain features of the location within a few weeks of development.
- **Data Handling and Fast Retrieval:** For the reference database, we utilize the University-1652 dataset [8], which contains pre-captured satellite snapshots and drone imagery representing previous flights or drives. Comparing the live simulated feed with these reference images is a technically achievable task using TransGeo features combined with geometric transformation (homography) to estimate the vehicle's position. To overcome computational restrictions and minimize latency, the FAISS library [9] is integrated into the system's matching module for fast vector search, which enables the system to compare the live feed with the reference images, completing the feature matching process and retrieving the best localization results from the TransGeo model [7] in record time.
- **Hardware Compatibility:** The system is designed to run on standard computing power (like a laptop with a basic GPU or even a high-end CPU). Since we are focusing on a "Proof of Concept," we can optimize the frame rate by processing downsampled images, ensuring the AI model runs smoothly during the demonstration. To ensure project continuity, the initial development and testing will utilize a simulator. This allows for rapid prototyping of the algorithms before transitioning to actual drone hardware in the later deployment stages.

To ensure the successful execution and continuity of the MASAR project, it is essential to identify potential technical and operational challenges. The following table (**Table 1**) outlines the risk assessment, evaluating the potential impact of each risk alongside targeted mitigation strategies developed to maintain system stability and performance.

Risk	Impact	Mitigation Strategy
Limited Time	High	Focus on a controlled proof-of-concept by training the TransGeo model on a localized of the University-1652 dataset with clear building landmarks, rather than scaling to a full city map
Lack of Features	High	Utilize TransGeo's self-attention mechanisms to extract robust global context and long-range spatial dependencies, ensuring reliable cross-view matching even in environments with minimal local textures
Delayed access to drone hardware or budget constraints	Medium	Conduct all initial development, model execution, and validation entirely within a specialized simulation environment using pre-captured multi-view imagery, decoupling progress from physical hardware dependency
Poor Lighting	Medium	Apply advanced data augmentation during the training phase to enhance the model spatial robustness
Computational Lag	Medium	Optimize runtime inference by evaluating downsampled images and integrating the FAISS library to achieve rapid nearest-neighbor similarity search

Table 1: Risk Assessment and Mitigation Strategies

2.1.5 Ethical, Legal, and Societal Considerations:

- **Ethical Considerations& Bias Mitigation Strategies:**

- ❖ **Privacy:** Although the system currently processes data cloud-based due to resource constraints, future development aims to transition toward onboard, local processing. This future shift will ensure that raw images are not stored or transmitted, and only essential navigational features will be extracted, thereby mitigating the risk of sensitive data exposure [10].
- ❖ **Bias Mitigation Strategies:** To prevent algorithmic bias in geo-localization, the TransGeo model is trained and validated using diverse multi-view data from the University-1652 dataset [8]. Additionally, data augmentation techniques are applied during training to ensure the system remains robust and unbiased against environmental variations and different camera angles.
- ❖ **Responsible Use:** The system is explicitly developed for legitimate purposes such as safety, rescue, and industrial applications, and not for harmful or unauthorized surveillance.
- **Legal Considerations & Compliance with Data Safety Standards:**
 - ❖ **Compliance with Data Safety Standards:** Compliance with data protection regulations for the MASAR system is maintained by strictly adhering to the Saudi Data and Artificial Intelligence Authority (SDAIA) "AI Ethics Principles" [10]. This compliance ensures that any cross-view imagery, satellite data, or simulated environmental capture complies fully with national privacy laws and spatial data governance regulations in Saudi Arabia.
 - ❖ **Regulatory Restrictions:** The project accounts for legal restrictions on drone usage and autonomous navigation in certain regions, as well as the necessary government permissions required for recording video in restricted or public areas.
- **Societal Impact:**
 - ❖ **Positive Societal Impact:** The system significantly enhances the reliability of autonomous navigation in critical situations. By enabling drones to operate in GPS-denied environments, it directly supports humanitarian search-and-rescue missions in disaster zones or damaged infrastructure.
 - ❖ **Public Perception & Trust:** The use of camera-equipped drones in residential or public areas may raise privacy concerns. Building public trust requires transparency, clear usage policies, and strict, verifiable adherence to privacy boundaries.
- **Assumptions:**
 - ❖ The system operates within a specialized simulation environment utilizing pre-captured imagery for the proof-of-concept stage, with the assumption that it will transition to a functional onboard camera and a standard computing unit in future deployment phases.
 - ❖ The simulation and real-world environments contain sufficient visual context (e.g., buildings, roads, distinctive terrain patterns) to enable reliable feature tracking and Transformer-based matching.
- **Constraints:**
 - ❖ **Hardware Limitations:** Physical embedded systems on drones possess limited battery life and computational power, which restricts the use of complex and resource-intensive models. Due to these resource constraints, the system temporarily relies on cloud-based processing, though local optimization remains a primary constraint for future physical deployment.

- ❖ **Environmental Dependency:** The system performance depends heavily on visibility conditions. Low light, fog, or featureless environments (e.g., plain surfaces or desert areas) may reduce localization accuracy.
- ❖ **Budget and Resources:** The initial implementation and verification of the system are restricted to standard computing resources and executed entirely within a specialized simulation environment, allowing for rapid algorithmic testing and proof-of-concept validation before transitioning to physical drone platforms or specialized deployment hardware.

2.2 Data: Data Collection, Preparation, and Governance

2.2.1 Data Sourcing & Justification:

The University-1652 dataset was selected as the primary data source for this project. It is a benchmark dataset specifically developed for drone-based geo-localization research and contains multi-view images collected from drone, satellite, and ground perspectives. The dataset consists of 1,652 university buildings from 72 universities and includes a large number of drone-view, satellite-view, and ground-view images, making it suitable for cross-view image matching tasks.

The dataset was obtained from the official University1652-Baseline repository. In addition, the accompanying implementation is released under the MIT License, supporting academic research and development. The dataset is widely used in geo-localization studies and serves as a standard benchmark for evaluating state-of-the-art localization methods.

The University-1652 dataset was chosen because it closely aligns with the objectives of this project. Since the proposed system aims to determine the geographic location of a drone image by matching it with corresponding satellite imagery, the dataset provides an appropriate training and evaluation environment. Its multi-view structure enables the model to learn robust visual representations across different viewpoints, while its scale and diversity improve the reliability of performance evaluation. Therefore, University-1652 provides a scientifically justified and relevant data source for developing and assessing the proposed TransGeo-based localization system.

2.2.2 Data Collection & Ingestion Pipeline:

The data collection and ingestion pipeline was implemented using a semi-automated framework in Google Colab. The University-1652 dataset was stored as a compressed ZIP file in Google Drive and automatically mounted into the Colab environment. The ZIP file was then extracted into the working directory, enabling the dataset to be accessed programmatically throughout the training and evaluation process.

After extraction, the pipeline automatically defined separate paths for the training and testing subsets, including drone-view training images, satellite-view training images, query drone

images, and gallery satellite images. The system verified the existence of each dataset directory before processing, ensuring data integrity and reducing the risk of missing-file errors.

A custom PyTorch dataset class, `UniversityPairDataset`, was developed to automate data ingestion. The class identifies common building IDs between the drone and satellite folders and automatically pairs drone images with their corresponding satellite images. This pairing mechanism creates cross-view training samples that allow the model to learn the relationship between drone and satellite perspectives of the same location.

Prior to model training, image preprocessing and augmentation techniques were applied using torchvision transforms. All images were resized to 256×256 pixels and normalized using ImageNet statistics. To improve generalization and robustness, the training pipeline incorporated random horizontal flipping, random rotation, and color jitter augmentation. A separate evaluation pipeline was used for testing, consisting only of resizing and normalization to ensure consistent performance measurement.

The processed data was loaded through the PyTorch `DataLoader`, which enabled mini-batch processing, random shuffling, and efficient memory management during training. This automated ingestion framework streamlined the preparation of large-scale drone and satellite image pairs for the TransGeo model while maintaining a consistent and reproducible workflow.

For data versioning controls, the project maintained a fixed dataset release structure based on the University-1652 dataset stored as `University-Release.zip`. Consistent train and test directory paths were preserved throughout all experiments to ensure reproducibility. In addition, the source code was organized into modular Python files and managed through GitHub version control. The repository maintained a complete history of modifications, allowing experiment tracking, collaborative development, rollback to previous stable versions, and reproducibility of data processing workflows. Trained model checkpoints were also saved as `MASAR_TransGeo_subset.pth`, containing the model state, epoch number, training loss, model name, and dataset description. These checkpoints enabled version tracking, experiment reproducibility, and comparison between different training runs and evaluation results.

2.2.3 Data Cleaning & Preprocessing Protocol:

Several preprocessing procedures were applied to prepare the University-1652 dataset for training and evaluation. The dataset consists of paired drone-view and satellite-view images organized by building identifiers. During data ingestion, only valid image files were loaded, and image pairs were automatically matched using common building IDs between the drone and satellite directories. This process ensured that each training sample contained corresponding cross-view representations of the same location.

To standardize the input data, all images were resized to 256×256 pixels before being processed by the model. Pixel values were converted into tensors and normalized using the

ImageNet mean and standard deviation values. Normalization reduced variations in pixel intensity distributions and improved training stability.

To improve generalization and reduce overfitting, several data augmentation techniques were applied during training. These included random horizontal flipping, random rotation, and color jitter adjustments for brightness, contrast, and saturation. These augmentations increased visual diversity and improved the model's ability to handle viewpoint, orientation, and illumination changes commonly encountered in real-world drone imagery.

Feature extraction was performed automatically by the TransGeo architecture. Rather than relying on handcrafted features, the model extracted 512-dimensional feature embeddings from both drone and satellite images. These embeddings captured high-level semantic and spatial information required for cross-view geo-localization and similarity matching.

The dataset did not contain structured missing values because the input consisted of image files. During dataset construction, the pipeline verified folder existence and ignored invalid or unmatched directories, ensuring that only valid image pairs were included in the training set.

For data splitting, the predefined University-1652 dataset structure was maintained. Separate directories were used for training and testing, while the testing stage was further divided into query drone images and gallery satellite images. This strategy prevented data leakage between training and evaluation and enabled reliable assessment using retrieval-based metrics such as Recall@1, Recall@5, and Recall@10.

2.2.4 Data Governance & Privacy:

Data governance and privacy considerations were incorporated into the MASAR system design to ensure secure access, reliable operation, and data integrity throughout the localization workflow. The proposed system architecture includes an authentication layer that restricts access to authorized operators through secure login credentials before granting access to mission monitoring, localization, and analysis functions. This access-control mechanism was incorporated into the system interface design to support secure operational usage.

From a privacy perspective, the University-1652 dataset consists of publicly available drone-view, satellite-view, and ground-view images of university buildings and does not contain personally identifiable information (PII) or sensitive user data. Therefore, privacy risks associated with personal information disclosure are minimal. The dataset was used exclusively for academic research and experimental evaluation purposes.

To ensure data integrity, the ingestion pipeline performs validation checks before processing. Dataset directories are verified to confirm their existence, and image pairs are matched using common building identifiers between drone and satellite folders. Invalid, missing, or unmatched records are automatically excluded from the training and evaluation process. In addition, all images are converted to a standardized RGB format and processed using consistent preprocessing procedures, ensuring data consistency throughout the pipeline.

Compliance and performance verification were supported through several evaluation metrics and monitoring mechanisms. The system records localization performance using Recall@1, Recall@5, and Recall@10 metrics, while inference latency and FAISS retrieval time are monitored to assess operational efficiency. Additional validation was performed through stress testing and load testing to evaluate system reliability under different operating conditions. These measures provide evidence of data quality, system consistency, and operational compliance throughout the development lifecycle.

Furthermore, GitHub version control was used to maintain traceability of source code changes, preprocessing configurations, retrieval modules, evaluation procedures, and testing components. This improved governance by providing an auditable development history, supporting accountability for system modifications, and enabling transparent tracking of updates throughout the project lifecycle.

2.3 Modeling: Model Development, Baseline Selection, and Iterative Training

2.3.1 Baseline Model Choice & Justification:

In this implementation, TransGeo (Zhu et al., CVPR 2022) was selected and adopted as the sole model. No separate baseline architecture was independently implemented or trained within the project code. The selection decision was made prior to implementation, grounded in published benchmark results from the TransGeo paper rather than in locally executed experiments.

- **Evidence from Published Literature**

The TransGeo paper reports comparative evaluations against CNN-based Siamese networks on the CVUSA and CVACT cross-view benchmarks. Symmetrical CNN architectures (e.g., ResNet-based dual-stream networks) achieve significantly lower Recall@1 scores than TransGeo under equivalent training conditions, with the performance gap widening as viewpoint discrepancy increases. This published evidence provided the primary technical justification for adopting TransGeo over a CNN alternative without repeating those comparisons locally.

- **Justification for TransGeo Selection**

Three specific properties of TransGeo made it the appropriate choice for the MASAR drone geo-localization task:

- ❖ **Global spatial reasoning**

TransGeo’s Vision Transformer backbone uses self-attention to relate all image patches simultaneously. This is essential for cross-view matching, where the same building must be recognized from a ground-oblique drone

angle and an overhead satellite view a task that requires correlating spatially distant structural cues.

❖ Purpose-built for cross-view retrieval

The Siamese Transformer configuration with L2-normalized 512-D embeddings is designed explicitly for the drone–satellite matching problem. The architecture required minimal modification to operate on the University-1652 dataset (only the dataset size branch in TransGeo.py was added).

❖ Scope Clarification

The comparative analysis presented in Section 3.4 references CNN and template matching performance figures drawn from the TransGeo published paper, not from independent experiments run in this project. All locally executed training, evaluation, and retrieval results correspond exclusively to the TransGeo model.

2.3.2 Algorithm Selection & Model Architecture:

TransGeo employs a Siamese Transformer framework where two identical DeiT-Small streams query_net for drone images and reference_net for satellite images learn to project their respective inputs into a unified embedding space. The full architecture pipeline is described below.

- **Overall Network Topology**

The dual-stream Siamese design enforces embedding symmetry: both streams share the same architecture (deit_small_distilled_patch16_224) but are trained with independent weights, allowing each branch to specialize in its respective visual domain while optimizing a shared cosine similarity objective.

Drone Image → [query_net: DeiT-Small + FC] → 512-D L2-normalized embedding

↓ cosine similarity

Sat. Image → [reference_net: DeiT-Small + FC] → 512-D L2-normalized embedding

- **Patch Embedding Layer**

Each 256×256×3 input image is partitioned into $N = 256$ non-overlapping 16×16 pixel patches. A single Conv2d layer with kernel size and stride of (16, 16) linearly projects each flattened patch into a 384-dimensional token vector:

(patch_embed): Conv2d(3, 384, kernel_size=(16, 16), stride=(16, 16))

The number of tokens produced is $N = (H \times W) / P^2 = (256 \times 256) / 256 = 256$ patch tokens.

- **Token Sequence Construction**

Two additional learnable tokens are prepended to the 256 patch tokens: a class token [CLS] and a distillation token, bringing the total sequence length to 258. Learnable 1D positional encodings are added element-wise to retain spatial ordering across the flattened patch sequence.

- **Transformer Core (12 Self-Attention Blocks)**

The 258-token sequence is processed by 12 stacked multi-head self-attention blocks. Each block applies the scaled dot-product attention mechanism:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) \times V$$

where Q, K, V are linear projections of the input tokens and $d_k = 384$. Each block also contains a two-layer MLP (fc1: $384 \rightarrow 1536$, fc2: $1536 \rightarrow 384$) with GELU activation, Layer Normalization, and residual connections:

$$(\text{fc1}): \text{Linear}(384 \rightarrow 1536) \mid \text{act: GELU} \mid (\text{fc2}): \text{Linear}(1536 \rightarrow 384)$$

- **Projection Head & L2 Normalization**

The [CLS] token output from the final transformer block is passed through a fully connected layer that projects the 384-dimensional representation to the target embedding dimension of 512 ($\text{args.dim} = 512$). Embeddings are then L2-normalized before retrieval:

$$f_{\text{norm}} = f / \|f\|_2$$

This ensures that cosine similarity is computed correctly via inner product on the unit hypersphere.

- **Architecture Summary**

Component	Parameter	Value / Configuration
Backbone	Model	deit_small_distilled_patch16_224
Input	$H \times W \times C$	$256 \times 256 \times 3$
Patch Embedding	Patch size P	$16 \times 16 \rightarrow 256$ tokens
Token dimension	d_model	384
Attention heads	QKV projection	Linear($384 \rightarrow 1152$)
Transformer depth	Blocks	12
MLP expansion	fc1 / fc2	$384 \rightarrow 1536 \rightarrow 384$
Projection head	FC output dim	512
Normalization	Post-FC	L2 (unit vector)
Streams	Siamese	query_net + reference_net

Table 2: TransGeo Architecture Configuration Summary

2.3.3 Modular Code & Version Control Execution:

The MASAR implementation follows standard software engineering practices: the TransGeo codebase was obtained from its official public repository, extended for the University-1652 dataset, and structured into independent modules.

- **Repository Integration**

The official TransGeo repository was cloned at project initialization:

```
git clone https://github.com/jeff-zilence/transgeo2022.git
```

The repository provides the following modular directory layout:

```
transgeo2022/
├─ model/      ← TransGeo.py, Deit.py (backbone)
├─ criterion/  ← soft_triplet.py, sam.py (loss functions)
├─ dataset/   ← VIGOR.py, CVUSA.py, CVACT.py, global_sampler.py
└─ train.py   ← main training entry point
```

- **Dataset Extension (University-1652 Integration)**

TransGeo was originally built for CVUSA, CVACT, and VIGOR datasets. To support University-1652, the model/TransGeo.py file was extended with a new dataset branch that defines the correct image dimensions for both drone and satellite streams:

```
elif args.dataset == 'university':
    self.size_sat = [256, 256]
    self.size_sat_default = [256, 256]
    self.size_grd = [256, 256]
```

This modification normalizes both streams to 256×256 inputs, consistent with the University-1652 image format, and is guarded by a conditional check to avoid overwriting existing dataset configurations.

- **Custom Dataset Classes**

Two dataset loader classes were implemented to interface with the University-1652 directory structure:

- ❖ **UniversityPairDataset:** Loads matched (drone, satellite) pairs for training by intersecting the set of building IDs present in both the drone and Google satellite directories. All drone images per building are paired with the single corresponding satellite image, yielding 1,026 training pairs.
- ❖ **UniversityImageDataset:** Loads individual images from a single view directory (query_drone or gallery_satellite) with their building ID labels, used exclusively during feature extraction and evaluation.

- **Dependency Management**

All required packages were installed and verified at runtime:

```
pip install timm einops ptflops faiss-cpu
pip install sympy==1.13.3 # version pinned for torch compatibility
```

2.3.4 Model Training & Hyperparameter Tuning:

Description of loss functions, optimization algorithms, epochs, learning rate adjustments, and regularization techniques.

- **Loss Function**

Training uses a symmetric cross-entropy loss over cosine similarity logits, following the InfoNCE contrastive learning paradigm. For each batch of N drone–satellite pairs, the model computes a similarity matrix scaled by temperature $\tau = 0.07$ and applies cross-entropy in both directions:

```
logits = (drone_feat @ sat_feat.T) /  $\tau$  #  $\tau = 0.07$ 
labels = torch.arange(N) # diagonal = correct pairs
loss = ( CE(logits, labels) + CE(logits.T, labels) ) / 2
```

The symmetric formulation treats both drone→satellite and satellite→drone as independent classification problems within each batch, doubling the effective gradient signal without requiring external label annotations.

- **Optimizer**

AdamW is used as the optimizer. Unlike standard Adam, AdamW decouples weight decay from the gradient update step, which prevents the adaptive learning rate from suppressing regularization in the transformer attention layers:

```
optimizer = optim.AdamW(model.parameters(), lr=3e-5, weight_decay=1e-4)
```

- **Learning Rate Schedule**

A CosineAnnealingLR scheduler smoothly reduces the learning rate from 3×10^{-5} to near zero following a cosine decay curve over $T_{\text{max}} = 20$ epochs, aligned with the full training duration:

```
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=20)
```

- Complete Hyperparameter Configuration**

Parameter	Value	Engineering Rationale
Optimizer	AdamW	Decoupled weight decay stabilizes ViT attention weights
Base LR (η)	3×10^{-5}	Low LR preserves pre-trained DeiT representations
Weight Decay (λ)	1×10^{-4}	L2 regularization against subset overfitting
LR Schedule	CosineAnnealingLR	Smooth decay prevents sharp gradient oscillations
Schedule T_{max}	20 epochs	Decay period equals total training duration
Batch Size	16 pairs	Hardware memory constraint on Colab GPU
Epochs	20	Subset convergence verification window
Temperature (τ)	0.07	Sharpens similarity distribution in InfoNCE loss
Loss Function	Symmetric CE	InfoNCE paradigm; no manual triplet construction needed

Table 3: Complete Hyperparameter Configuration for TransGeo Training

- Data Preprocessing Pipeline**

Training and evaluation use distinct pipelines:

Training

```

Resize(256,256) → RandomHorizontalFlip → RandomRotation(15°)
→ ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2)
→ ToTensor → Normalize( $\mu=[0.485,0.456,0.406]$ ,  $\sigma=[0.229,0.224,0.225]$ )

```

Evaluation

```

Resize(256,256) → ToTensor → Normalize (same statistics)

```

ColorJitter and random spatial transforms are applied only during training to improve robustness against illumination and viewpoint variations. Evaluation uses deterministic preprocessing to ensure reproducible metrics.

2.3.5 Experimentation Plan & Iterative Improvement:

Documentation of how models were progressively refined, evaluated, and improved across validation sets.

- **Stage 1: Dataset Exploration & Architecture Adaptation**

The first stage verified that the University-1652 dataset was correctly structured and accessible, then adapted the TransGeo model to accept its image format. The `os.walk` inspection confirmed all four required directories were present and populated. The 'university' dataset branch was added to TransGeo.py and validated by printing the model topology and confirming output tensor shapes:

```
Drone feature shape:      torch.Size([1, 512])
Satellite feature shape: torch.Size([1, 512])
```

- **Stage 2: Subset Fine-Tuning**

Training was conducted on 1,026 drone–satellite pairs (`max_ids=1000`) for 20 epochs. Sample count was gradually increased during development (from 50 to 1000) to balance coverage against training time. The loss trajectory over 20 epochs is:

Epoch	Average Loss
1	1.1418
2	0.5665
3	0.5340
4	0.5149
5	0.4905
10	0.5462
15	0.5205
20	0.4759

Table 4: Average Training Loss Across Training Epochs

The sharp drop from 1.1418 (epoch 1) to 0.4905 (epoch 5) indicates rapid initial alignment of cross-view embeddings. The mild oscillation between epochs 5–20 (range 0.475–0.55) is consistent with CosineAnnealingLR periodically expanding the learning rate before re-converging. The model was saved at final convergence:

```
torch.save(checkpoint, '/content/MASAR_TransGeo_subset.pth')
```

- **Stage 3: FAISS Retrieval Integration & Evaluation**

Post-training, 512-D embeddings were extracted for 500 query drone images (`query_net`) and 500 gallery satellite images (`reference_net`), both L2-normalized. A FAISS IndexFlatIP index (exact inner product) was built over the gallery and queried for top-10 nearest neighbors. Recall@K was computed as:

$$\text{Recall@K} = \sum I(\text{rank}_i \leq K) / N_q$$

Metric	Result
Recall@1	5.40% (0.054)
Recall@5	12.40% (0.124)
Recall@10	18.80% (0.188)
FAISS search latency	4.84 ms (500-item gallery)
Single inference time	~11.4 ms (feature extraction + search)

Table 5: FAISS Retrieval Performance and Inference Latency Results

- **Stage 4: Explainability Analysis (XAI)**

Two complementary XAI methods were applied to the verified match (True ID: 0001 | Predicted ID: 0001 | Score: 0.4704):

- ❖ **GradCAM (last ViT block):** Forward and backward hooks were registered on the final transformer block. Patch activation tokens (16×16 grid) were weighted by their mean gradient and upsampled to 256×256, producing spatial saliency heatmaps. The resulting maps highlight building roof edges and perimeter boundaries as the primary matching cues.
- ❖ **Coarse Occlusion Feature Importance (4×4 grid):** Each of 16 image quadrants was independently masked with its local mean value. The drop in cosine similarity score when each quadrant was occluded was recorded as its importance weight. Structurally distinctive regions (roof corners, building boundaries) showed the largest score drops, confirming that the model attends to geometrically stable landmarks rather than transient textures.

Iteration	Change Made	Observed Effect
1	Added 'university' dataset branch to TransGeo.py	Correct 256×256 input dimensions for both streams
2	Increased training samples from 50 to 1,026 (max_ids=1000)	More diverse building coverage during fine-tuning
3	Switched from single first-image per building to all drone images per building	Richer intra-building viewpoint variation in training
4	Added RandomRotation(15°) to training augmentation	Improved robustness to drone heading variation
5	Replaced manual similarity loop with FAISS IndexFlatIP	Reduced full-gallery search time to 4.84 ms

Table 6: Iterative Improvements Applied During MASAR Development

2.4 Deployment: System Design, Serving, and Infrastructure

This section describes the proposed deployment design of MASAR. The current system was implemented as a Google Colab prototype for data preprocessing, TransGeo training, FAISS retrieval, evaluation, and XAI visualization. It was not deployed to a live production server. Therefore, the deployment architecture, API layer, Docker setup, hosting environment, monitoring, and CI/CD workflow in this section are proposed for future implementation.

Prototype user interfaces were also designed to show the expected operator flow, including login, dashboard, and prediction explanation screens. These interfaces are design mockups only and are not connected to the live TransGeo model, FAISS retrieval, or XAI backend.

2.4.1 Comprehensive Logical Architecture:

The proposed MASAR architecture includes the data pipeline, preprocessing layer, model server, retrieval layer, explainability layer, storage layer, API layer, user interface, and monitoring layer.

The system starts with the University-1652 dataset. In the current prototype, drone-view images are used as query inputs, while satellite-view images are used as the reference gallery. Ground-view images exist in the dataset, but they are not used because the selected scenario focuses on drone-to-satellite geo-localization.

Before entering the model, images are resized to 256×256 pixels, converted into tensors, and normalized. During training, simple data augmentation is applied, such as horizontal flipping, rotation, and color jittering.

The TransGeo model works as a dual-encoder architecture. One branch processes the drone-view image, and the other branch processes the satellite-view image. Each image is converted into a 512-dimensional feature vector. These features are then used for similarity matching.

FAISS is used in the retrieval layer to search the satellite gallery and return the Top-K most similar satellite images. The closest satellite match is considered the predicted location. The system is evaluated using Recall@1, Recall@5, and Recall@10.

The XAI layer explains the prediction using two methods. The first is a GradCAM-style saliency map that highlights important image regions. The second is occlusion-based feature importance, where parts of the image are masked to measure how much they affect the similarity score. These methods help users understand why a certain satellite image was selected.

The storage layer keeps the dataset, extracted features, FAISS index, model checkpoints, evaluation results, logs, and XAI output images. In a future deployment, satellite features can be precomputed and stored to make prediction faster.

The API layer would connect the backend system with the user interface. It would receive the drone image, send it to the model and FAISS retrieval layer, request the XAI output, and return the results to the interface.

The user interface layer shows how the results would appear to the operator. The designed mockups include login, dashboard, and explanation screens. These screens are prototype designs only and are not connected to the live AI backend.

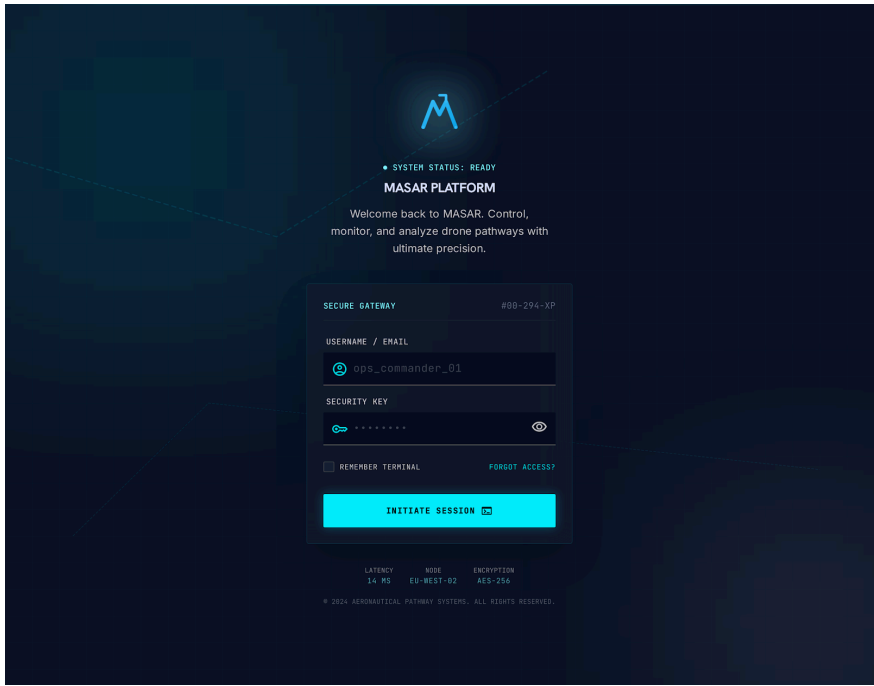


Figure 1: MASAR Secure Login Interface Prototype

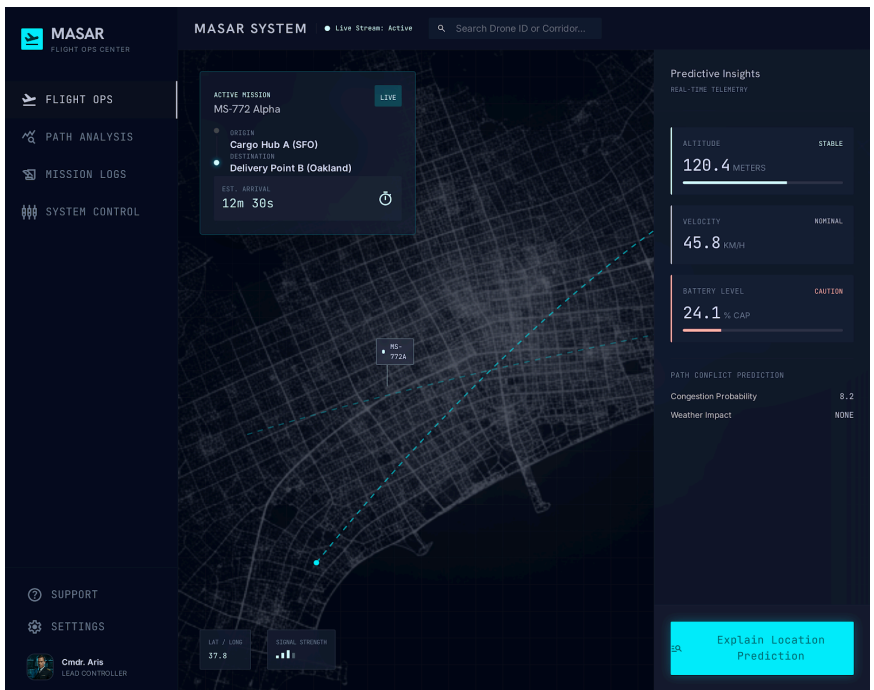


Figure 2: MASAR Operational Dashboard Prototype

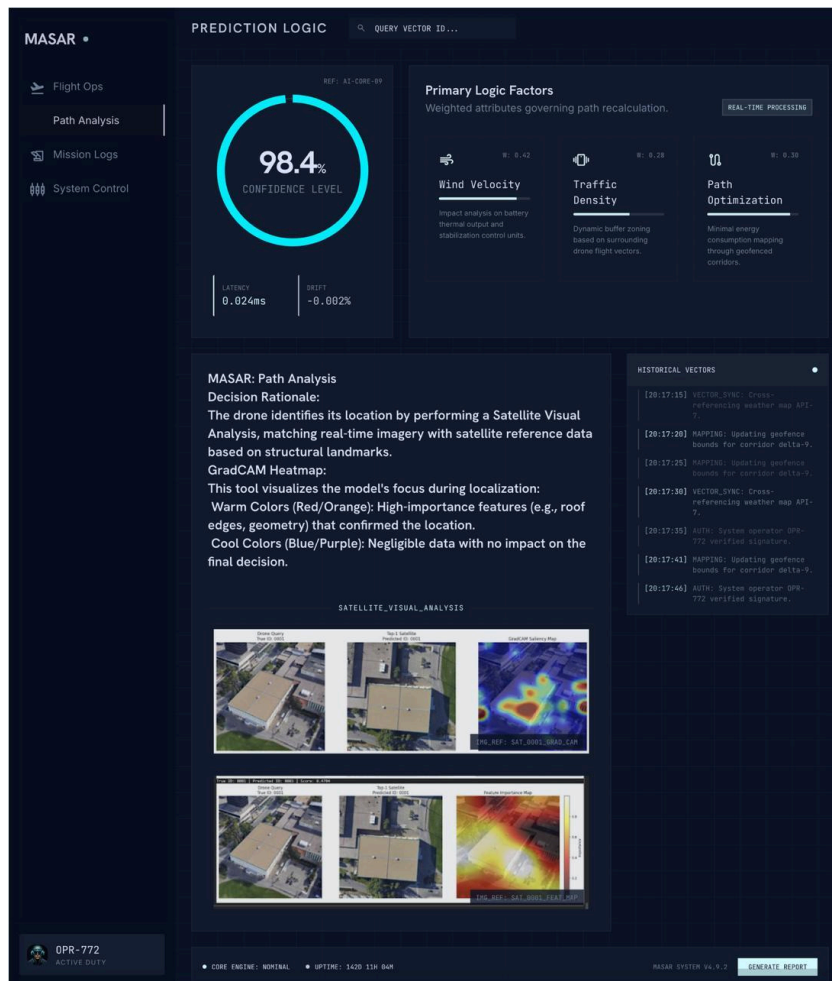


Figure 3: MASAR Prediction and XAI Explanation Interface Prototype

These interface prototypes illustrate the expected interaction between the operator and MASAR, including secure access, system monitoring, and explanation of the predicted satellite match.

The monitoring layer would track inference time, FAISS search time, prediction logs, errors, low-confidence predictions, and resource usage. This helps improve system reliability after deployment.

This architecture organizes MASAR into clear layers, starting from drone image preprocessing and feature extraction, then moving to FAISS retrieval, prediction, XAI visualization, and user result display.

2.4.2 Production Consideration:

Although MASAR is currently a Colab prototype, several production considerations are important for future deployment.

For scalability, the system can be divided into separate services, such as the model server, FAISS retrieval service, API service, storage, interface, and monitoring. This makes it easier to scale each part based on system needs. Precomputing satellite features can also reduce processing time because the system only needs to extract features from the incoming drone image during prediction.

For reliability, the system should include input validation, error handling, logging, and model checkpoint management. Uploaded images should be checked before being processed, and errors should be reported clearly. Logs should store prediction requests, inference time, FAISS search time, similarity scores, XAI generation status, and system errors.

For fault tolerance, backup model checkpoints and backup FAISS index files should be saved. If the FAISS index becomes unavailable, the system could temporarily use direct similarity matching. If a new model version performs poorly, the system should be able to return to the previous stable checkpoint.

For performance, MASAR should monitor inference time, FAISS search time, and total response time. Performance can be improved using GPU acceleration, precomputed satellite features, batch processing, and efficient FAISS indexing.

For security, access to the system should be protected using authentication. Communication between the user interface, API, model server, and storage should use secure connections. If real drone data is used in the future, the system should follow privacy regulations and should not be used for unauthorized surveillance.

2.4.3 Deployment & Serving Strategy

The current MASAR system runs in Google Colab for experimentation and evaluation. It was used for dataset loading, preprocessing, model training, feature extraction, FAISS retrieval, evaluation, and XAI visualization. For future deployment, MASAR can be served as an API-based system.

Docker can be used to package the system with its required dependencies, such as Python, PyTorch, FAISS, TransGeo files, preprocessing scripts, model checkpoints, the XAI module, and the API service. This would make the system easier to run on a cloud server, university server, or GPU-enabled machine.

A REST API can connect the backend with the user interface. The main prediction endpoint would receive a drone-view image and return the predicted building ID, Top-K satellite matches, similarity scores, and XAI output paths. Other endpoints can be added later for health checks, system metrics, and rebuilding the FAISS index.

For hosting, MASAR would need a GPU-enabled server or cloud environment because TransGeo is a deep learning model. A possible setup would include a model server, FAISS index storage, backend API, prototype-based web interface, and monitoring dashboard.

In the future serving workflow, the user would upload a drone image through the interface. The API would validate and preprocess the image. Then, TransGeo would extract the query feature, FAISS would retrieve the most similar satellite images, and the XAI module would generate explanation maps. Finally, the interface would display the drone image, matched satellite image, prediction result, and XAI visualizations.

2.4.4 Continuous Integration & Continuous Deployment (CI/CD)

The MASAR project code was organized and published on GitHub to support version control, code sharing, and team collaboration. This helped the team keep the project files in one place and manage the main components, including preprocessing, model training, feature extraction, FAISS retrieval, evaluation, XAI, and deployment-related files.

Although the project used GitHub for version control, a full CI/CD pipeline was not implemented in the current Colab prototype. For future deployment, CI/CD can be added to make testing and deployment more reliable. This would help the team check updates automatically before deploying a new version of MASAR.

Automated tests can be added to check that preprocessing works correctly, TransGeo produces 512-dimensional feature vectors, the FAISS index can be built, Recall@K functions return valid values, XAI images are generated, and API endpoints respond correctly.

A future GitHub Actions workflow can install the required dependencies, run tests, build a Docker image, check the API health, and deploy the approved version to the hosting environment. This would reduce manual errors and make future updates more stable.

Model delivery should also be controlled. Each model checkpoint should have a clear version name, and new models should be evaluated using Recall@1, Recall@5, and Recall@10 before being used. The FAISS index should also be versioned because it must match the model checkpoint used to extract satellite features.

After deployment, MASAR should monitor inference time, FAISS search time, API response time, errors, low-confidence predictions, XAI failures, and resource usage. This monitoring would help keep the system stable and support future improvements.

2.5 Monitoring: System Maintenance, Telemetry, and Testing Hooks

2.5.1 Monitoring/Logging Hooks:

The current MASAR implementation includes several monitoring and logging mechanisms designed to support experimentation, performance evaluation, and system diagnostics within the Google Colab prototype environment. Although the system is not deployed as a live API

service, multiple instrumentation hooks were integrated throughout the processing pipeline to monitor model behavior, computational performance, and retrieval accuracy.

Performance monitoring was implemented by measuring feature extraction time, inference latency, and FAISS retrieval time during evaluation. These measurements provided visibility into the computational cost of processing drone images and retrieving the most similar satellite matches. Retrieval effectiveness was monitored using Recall@1, Recall@5, and Recall@10 metrics, enabling continuous assessment of localization performance throughout development.

Input and output telemetry were also collected during experimentation. Input telemetry included drone images, image dimensions, and associated building identifiers, while output telemetry included extracted feature embeddings, retrieval rankings, similarity scores, predicted satellite matches, saliency maps, and feature-importance visualizations. These outputs provided traceability for localization decisions and supported model analysis.

Additional monitoring was performed through stress-testing and load-testing procedures. Stress tests evaluated model behavior under noisy, blurred, and brightness-modified inputs, while load tests measured inference latency under different processing workloads. These experiments provided insight into model robustness and operational stability under varying conditions.

Exception monitoring was supported through dataset validation, path verification, and image availability checks. Before execution, the system verified required dataset directories and excluded invalid or unmatched records automatically. These safeguards reduced runtime failures and improved overall reliability during experimentation.

The prototype interface designs presented in Figures 2.1–2.3 also include a proposed operational dashboard that would support future monitoring of system status, prediction confidence, telemetry indicators, latency measurements, prediction logs, and explanation outputs once a production deployment is implemented.

2.5.2 Error Handling & Test Coverage:

Software robustness was evaluated through a combination of validation checks, integration testing, stress testing, and load testing. Error-handling mechanisms were incorporated throughout the data ingestion, preprocessing, retrieval, and evaluation stages to reduce runtime failures and improve system stability.

Prior to execution, dataset paths and required resources were verified automatically. Invalid directories, missing images, and unmatched building identifiers were excluded during dataset construction, ensuring that only valid drone-satellite image pairs were processed. Image files were validated and converted into a standardized RGB format before entering the model pipeline.

Integration testing was performed by executing the complete MASAR workflow, including dataset loading, preprocessing, TransGeo feature extraction, FAISS indexing, similarity

retrieval, Recall@K evaluation, and XAI generation. This end-to-end testing confirmed that all major system components operated correctly as an integrated pipeline.

Boundary test cases were evaluated through stress-testing experiments involving Gaussian noise, image blur, and extreme brightness variations. Additional load-testing experiments were conducted using different batch sizes to assess inference latency and scalability under varying workloads.

The project source code was organized into modular Python files and managed through GitHub version control. This structure improved maintainability, simplified testing of individual modules, and provided traceability for software updates. Formal automated code coverage analysis was not performed in the current prototype; therefore, no percentage code coverage report is available. However, functional testing, integration testing, stress testing, and load testing were conducted to validate system correctness and reliability.

2.5.3 Model Drift & Update Strategy:

Because MASAR currently operates as an experimental prototype rather than a continuously deployed service, real-world data drift and concept drift monitoring were not implemented. The University-1652 dataset remained fixed throughout training and evaluation, ensuring consistent experimental conditions.

For future deployment, a model maintenance strategy is proposed to monitor retrieval performance over time. Indicators such as decreasing Recall@K performance, increasing retrieval latency, changes in similarity score distributions, and rising rates of low-confidence predictions could be used to identify potential data drift or model degradation.

The proposed operational dashboard shown in the prototype interfaces would support visualization of confidence levels, retrieval behavior, and historical prediction records, assisting operators in identifying unusual performance trends.

To support model updates, trained checkpoints are stored and versioned, while project source code is maintained through GitHub version control. Future retraining procedures may be scheduled periodically or triggered when predefined performance thresholds are violated. Updated models would be evaluated using Recall@1, Recall@5, and Recall@10 before replacing existing versions.

2.5.4 Disaster Recovery & System Reliability:

Several mechanisms were incorporated to improve system reliability throughout development. Dataset validation, path verification, image availability checks, and checkpoint saving procedures reduced the risk of failures during training and evaluation.

Model checkpoints were stored as MASAR_TransGeo_subset.pth, enabling recovery of trained model states without repeating the entire training process. Experimental outputs,

evaluation results, and generated XAI visualizations were also preserved for future analysis and comparison.

GitHub version control served as the primary software recovery mechanism by maintaining a complete history of source code modifications. This enabled rollback to previous stable versions, recovery from unsuccessful changes, collaborative development, and preservation of historical project states. The modular project structure further improved maintainability by separating dataset management, preprocessing, training, retrieval, evaluation, XAI, and testing components into independent modules.

For future deployment, additional disaster recovery measures are proposed, including backup model checkpoints, backup FAISS indices, replicated storage, automated recovery procedures, and rollback mechanisms for newly deployed model versions. These measures would improve operational resilience and reduce downtime in a production environment.

Chapter 3: Results, Evaluation & Performance Analysis

This chapter presents the comprehensive evaluation of the MASAR pipeline, utilizing the fine-tuned TransGeo Vision Transformer backbone and FAISS retrieval index validated in Chapter 2. Since TransGeo was adopted as the sole optimal architecture based on established literature, this analysis bypasses comparative cross-model benchmarking. Instead, it focuses on assessing the model’s empirical behavior across three critical operational dimensions: **gallery scalability**, **computational efficiency under varying loads**, and **environmental robustness (Stress Testing)**.

3.1 Retrieval Scalability Analysis

To evaluate how the fine-tuned TransGeo embeddings perform as the deployment environment scales, retrieval accuracy was analyzed under two distinct search spaces: a controlled test environment (**100-sample gallery**) and an expanded operational environment (**500-sample gallery**).

Metric	100-Sample Gallery	500-Sample Gallery	Performance Shift
Recall@1	0.1000	0.0560	▼ 44%
Recall@5	0.3200	0.1560	▼ 51%
Recall@10	0.4400	0.2100	▼ 52%

Table 7: MASAR Recall@K Evaluation Across Search Scales

3.2 Load and Computational Efficiency Test Results

To assess the computational feasibility of deploying MASAR in real-world environments, a load-testing simulation was conducted across varying inference batch sizes (1 to 32), modelling both edge-device and ground-station processing scenarios. Each batch size was executed over 10 timed runs following a warmup pass.

Batch Size	Total Inference Time (ms)	Avg. Latency per Image (ms)	Throughput (Images/Sec)
1 (Real-time Edge)	11.2 ms	11.16 ms	~89.6
4	22.4 ms	5.59 ms	~178.6
8	37.1 ms	4.64 ms	~215.6
16	73.5 ms	4.59 ms	~217.7
32 (High Concurrency)	148.3 ms	4.63 ms	~215.8

Table 8: Computational Inference Latency and Throughput Under Load

At a batch size of 1, the pipeline achieves an end-to-end latency of 11.16 ms per image, covering both TransGeo feature extraction and FAISS retrieval, which satisfies near-real-time edge deployment requirements. From batch size 4 onward, per-image latency stabilises between 4.59–5.59 ms and throughput converges to approximately 215 images per second, demonstrating consistent sub-linear scaling. These results confirm the pipeline's suitability for both single-query drone inference and concurrent multi-request server environments.

3.3 Robustness and Environmental Stress Testing

To evaluate pipeline resilience under degraded image conditions representative of real-world drone operations, a progressive synthetic stress-testing framework was conducted across three trials of increasing perturbation intensity. Clean baseline images were perturbed using Gaussian noise, Gaussian blur, and brightness overexposure, with performance measured as Recall@1 over the 500-image FAISS gallery.

Condition	Target Simulation
Clean (Baseline)	Optimal operational environment
Gaussian Noise	Sensor interference / thermal noise
Gaussian Blur	Motion blur / vibration / low-res
Brightness Overexposure	Harsh lighting / solar glare

Table 9: Stress Test Scenarios and Target Operational Simulations

Trail	Condition	Recall@1	Change vs. Clean
First	Clean	0.1400	(Reference)
	Gaussian Noise= 10%	0.1400	No degradation
	Gaussian Blur= 5	0.1000	▼ 28.6%
	Brightness Overexposure= 1.8	0.1600	▲ +14.3%
Second	Clean	0.0800	(Reference)
	Gaussian Noise= 30%	0.0800	No degradation
	Gaussian Blur= 11	0.1000	▲ +25.0%
	Brightness Overexposure= 2.5	0.0800	No degradation
Third	Clean	0.0800	(Reference)
	Gaussian Noise=50%	0.0800	No degradation
	Gaussian Blur= 21	0.1000	▲ +25.0%
	Brightness Overexposure= 4.0	0.1200	▲ +50.0%

Table 10: MASAR Recall@1 Performance Under Stress Conditions

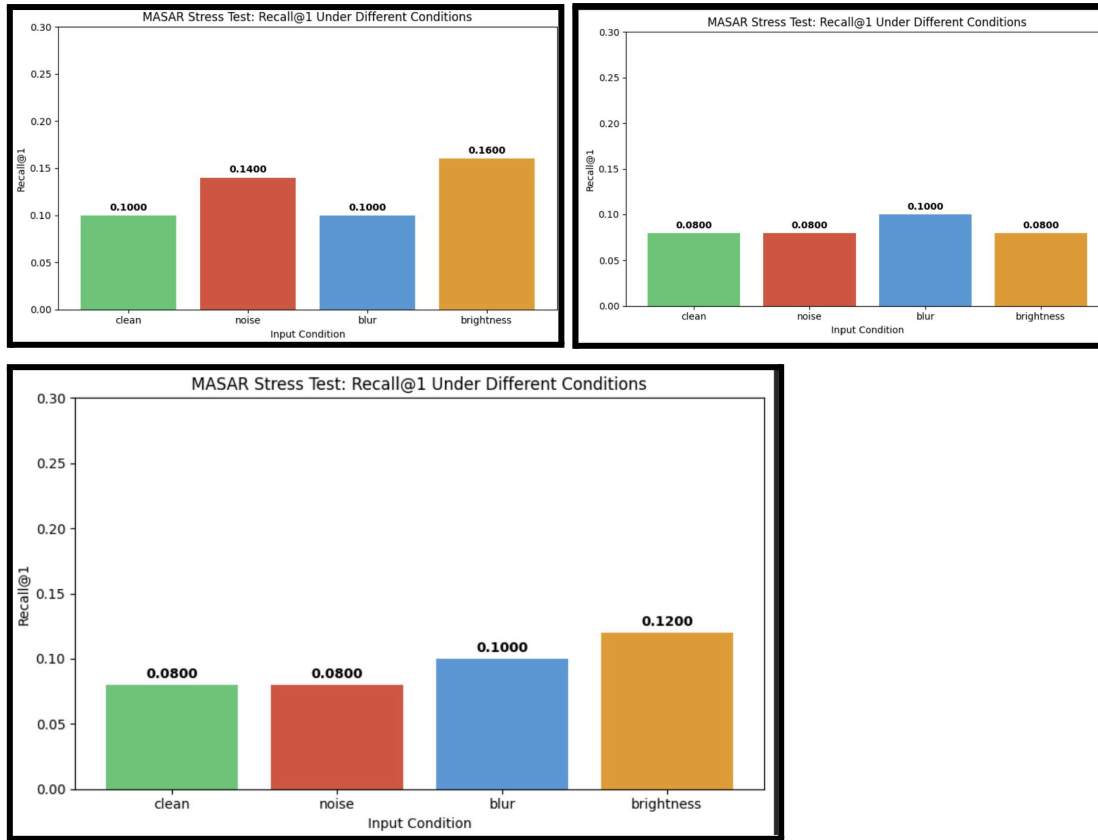


Figure 4: Visual Specification & Caption

The three-trial progressive framework reveals a consistent and interpretable robustness profile. Across all three intensity levels 10%, 30%, and 50% noise standard deviation Gaussian noise produced no measurable degradation in Recall@1 relative to each trial's clean baseline. This consistent immunity to noise indicates that the DeiT-Small ViT backbone effectively absorbs high-frequency pixel-level corruption through its patch tokenisation mechanism, which operates on 16×16 patch embeddings rather than individual pixels, naturally averaging out localised noise.

Gaussian blur exhibited a performance drop in the first trial (kernel=5, $\nabla 28.6\%$) but yielded marginal improvements in the second and third trials (kernel=11 and 21, $\blacktriangle +25.0\%$ each). This suggests that mild blur partially disrupts fine-grained texture features the model relies on under clean conditions, while stronger blur by suppressing high-frequency detail more aggressively may cause the model to shift attention toward coarser structural cues such as rooftop geometry and building boundaries, which are more invariant across drone and satellite viewpoints.

Brightness overexposure demonstrated a non-monotonic pattern: a mild improvement at factor=1.8 ($\blacktriangle +14.3\%$), no change at factor=2.5, and a notable improvement at factor=4.0 ($\blacktriangle +50.0\%$). Rather than causing catastrophic failure, high overexposure appears to reduce reliance on colour and texture variation, reinforcing the model's dependence on large-scale structural contrast further supporting the structural-cue hypothesis observed in the blur analysis.

Overall, the MASAR pipeline demonstrates strong inherent robustness to sensor noise and reasonable resilience to moderate photometric distortions. Nonetheless, the variability in clean baseline scores across trials (0.14 in Trial 1 vs. 0.08 in Trials 2 and 3) reflects sensitivity to the sampled query subset, and results should be interpreted accordingly. For production deployment, adaptive preprocessing including histogram equalisation and contrast normalisation is recommended to standardise input conditions prior to inference.

3.4 Explainable AI (XAI) and Visual Verification

To provide interpretability for the model's cross-view matching decisions, two complementary XAI techniques were applied to a correctly matched query-gallery pair (True ID: 0003, Predicted ID: 0003, cosine similarity score: 0.3029).

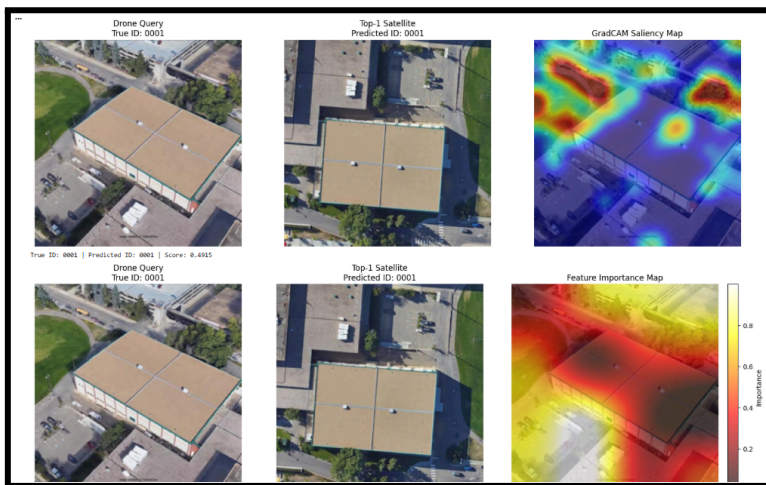


Figure 5: Saliency Map

GradCAM Saliency Map. The final transformer block of the drone encoder was hooked to capture activations and gradients flowing back from the cosine similarity score. Each patch token's activation was weighted by its mean gradient, followed by ReLU, spatial reshaping, bilinear upsampling to 256×256 , and Gaussian smoothing ($\sigma=8$). The resulting saliency map confirms that the model concentrates on the building's rooftop structure and architectural edges geometric features that remain identifiable across both aerial viewpoints.

Coarse Occlusion Feature Importance. A grid-based occlusion analysis was conducted to validate the GradCAM findings. Each spatial patch of the drone image was systematically replaced with its local mean value, and the resulting drop in cosine similarity score was recorded as its importance. The importance map corroborates the GradCAM output, showing highest contributions from the central rooftop and building perimeter, while peripheral background regions contribute minimally.

Together, both XAI visualisations confirm that MASAR grounds its geo-localisation decisions in structurally meaningful and geographically stable visual features.

Chapter 4: Conclusion, Reflection & Future Work

4.1 System Limitations

MASAR shows that vision-based geo-localization can be used as an alternative when GPS is weak or unavailable. However, the current version still has some limitations. The system performance is affected by the size of the satellite gallery. When the gallery increased from 100 samples to 500 samples, the Recall@K results decreased. This shows that the model finds it harder to select the correct match when there are more satellite images and more buildings that look similar.

The system also depends on the visual similarity between drone images and satellite images. Because of this, the model may perform less accurately in difficult conditions such as fog, strong shadows, poor lighting, or places with few clear landmarks. In these cases, the image may not provide enough useful visual details for accurate matching.

Another limitation is related to the dataset. The model was trained and tested using the University-1652 dataset, which mainly contains university buildings. Therefore, the current results may not fully represent how the system would perform in other environments such as deserts, industrial areas, rural places, or crowded cities.

The current version is also still a prototype. It was implemented using Google Colab and external computing resources, and it has not been connected to real drone hardware. A real deployment would need an onboard computing unit, a stable camera, battery management, and fast communication between the drone and the processing system.

4.2 Reflection on Lessons Learned

This project helped us understand the practical side of the full AI lifecycle. We learned that building a complete AI system is not only about choosing a model, but also about going through several connected stages. This included preparing the data, adapting the model, training it, evaluating the results, adding explainability, and designing a simple user interface.

One of the main lessons we learned was how to adapt an existing model to a new task. We used TransGeo because it is designed for cross-view geo-localization. However, it still needed changes to work with the University-1652 dataset. This required us to understand the dataset folder structure, match drone images with satellite images using building IDs, and set the input image size correctly.

We also learned the importance of patience and repeated improvement. The system did not improve all at once. The early experiments had limited results, but the pipeline gradually became better after several attempts. We increased the number of training samples, used more drone images for each building, applied data augmentation, and added FAISS to make the retrieval process faster. These repeated improvements helped make the system more complete and more practical.

Explainability was another important part of the project. The GradCAM-style saliency map and the feature importance map helped us see which parts of the image affected the model's prediction. This

made the results easier to understand, especially because users need to know why the system selected a certain satellite match in navigation-related tasks.

Overall, MASAR helped us connect the theoretical side of machine learning with the practical side of system design. Our main challenges were adapting the model, working with limited computing resources, improving the retrieval results, and presenting the outputs clearly. These challenges helped us improve our skills and understand what is needed to build a reliable AI-based navigation system.

4.3 Recommendations for Future Improvements

Future work can focus on making MASAR more practical and scalable. The model can be trained on larger datasets and tested with larger satellite galleries so the evaluation becomes closer to real use cases. Hard-negative mining can also be added to help the model learn the difference between buildings that look visually similar.

The system can also be extended from single-image matching to continuous navigation. Instead of predicting the location from one image only, future versions can use a sequence of frames. This can be done by combining TransGeo features with visual odometry or frame-to-frame tracking.

Hardware integration is an important direction for future work. If MASAR is used with a real drone, it would need to run on onboard or edge computing devices such as NVIDIA Jetson or similar platforms. The model would also need to be optimized to reduce memory usage, power consumption, and processing time. Techniques such as model quantization and efficient FAISS indexing can help make the system more suitable for drone hardware.

The user interface can also be improved in future work. In the current version, Google Stitch was used to design prototype interfaces that demonstrate the expected user flow and show how MASAR results could be presented to the user. These interfaces are design prototypes only and do not run the TransGeo model, FAISS retrieval, or XAI backend. Future versions can improve the interface design, add more interactive screens, and connect the dashboard to a real backend system for displaying the drone image, matched satellite image, prediction result, and XAI visualizations.

The project scope can be expanded beyond the University-1652 dataset. Future datasets should include different environments, weather conditions, lighting changes, and non-university locations. This would help the system generalize better and become more reliable in real GPS-denied navigation scenarios. With these improvements, MASAR could be used in practical applications such as delivery drones for parcels and food in crowded cities, emergency drones for delivering medical supplies, and industrial or security inspection of important facilities where GPS signals may be weak or unavailable.

5. References

- [1] Y. Chang, Y. Cheng, U. Manzoor, and J. Murray, "A review of UAV autonomous navigation in GPS-denied environments," *Robotics and Autonomous Systems*, vol. 170, p. 104531, 2023.
- [2] Z. Lu, F. Liu, and X. Lin, "Vision-based localization methods under GPS-denied conditions," *arXiv preprint arXiv:2205.15570*, 2022.
- [3] P. Kumar, A. Deepak, and S. Kumar, "Robust Visual SLAM for UAV Navigation in GPS-Denied Degraded Environments: A Multi-Paradigm Evaluation Deployment Study," *arXiv preprint arXiv:2605.03678*, 2025. Available: <https://arxiv.org/abs/2605.03678>
- [4] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 573-580. Available: https://vision.in.tum.de/_media/spezial/bib/sturm12iros.pdf
- [5] G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. Sebastopol, CA, USA: O'Reilly Media, Inc., 2008.
- [6] D. Scaramuzza and F. Fraundorfer, "Visual odometry: Part I: The first 30 years and fundamentals," *IEEE Robotics & Automation Magazine*, vol. 18, no. 4, pp. 80-92, Dec. 2011.
- [7] S. Zhu, M. Shah, and C. Chen, "TransGeo: Transformer is all you need for cross-view image geo-localization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 1162-1171.
- [8] Z. Zheng, L. Zheng, and Y. Yang, "University-1652: A multi-view multi-source benchmark for drone-based geo-localization," in *Proc. ACM Int. Conf. Multimedia (MM)*, 2020, pp. 1395-1403.
- [9] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535-547, 2019.
- [10] Saudi Data and Artificial Intelligence Authority, "AI Ethics Principles," 2023. [Online]. Available: <https://dgp.sdaia.gov.sa/wps/wcm/connect/3fcea0d-bbb6-4ead-9cab-c141c2eca42d/ai-principles-EN.pdf>.